Analiza sentymentu na podstawie recenzji aplikacji Spotify

Cel analizy

Otrzymanie modelu, który na podstawie treści recenzji będzie potrafił poprawnie ją zaklasyfikować do jednej z trzech klas opinii:

- pozytywne
- neutralne
- negatywne

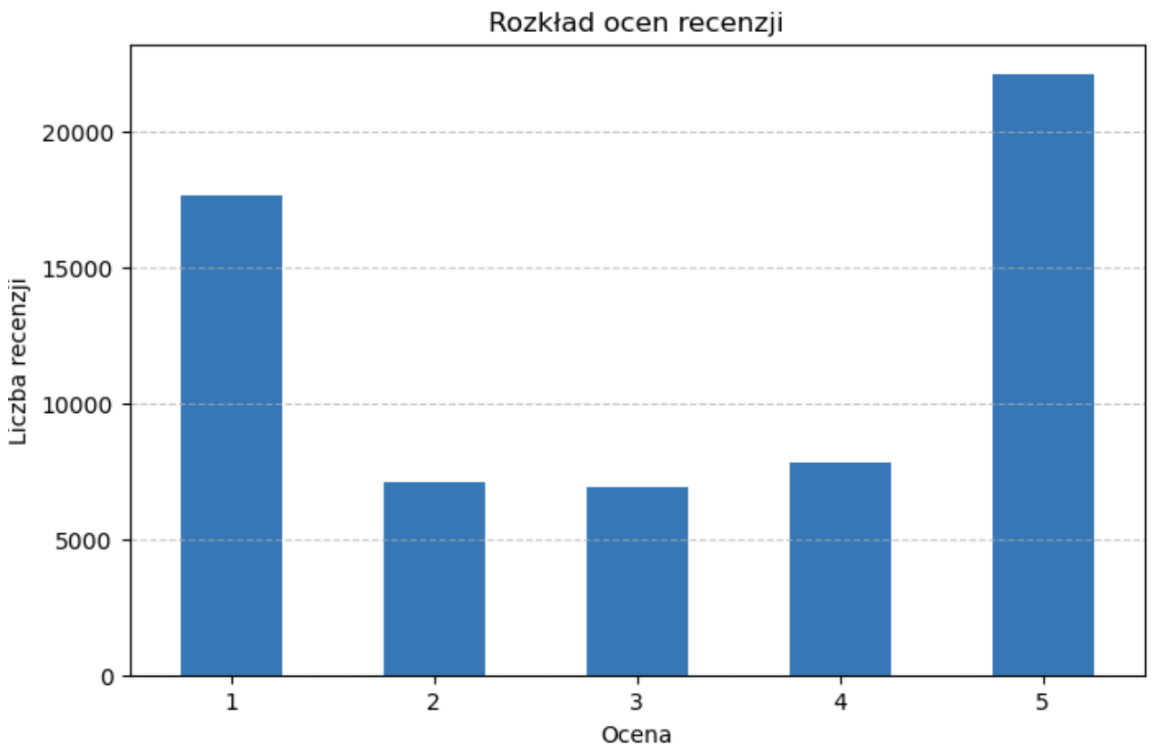
Zbiór danych

- Zbiór danych użytych do analizy
- Usunięcie kolumn: Time_submitted, Total_thumbsup, Reply

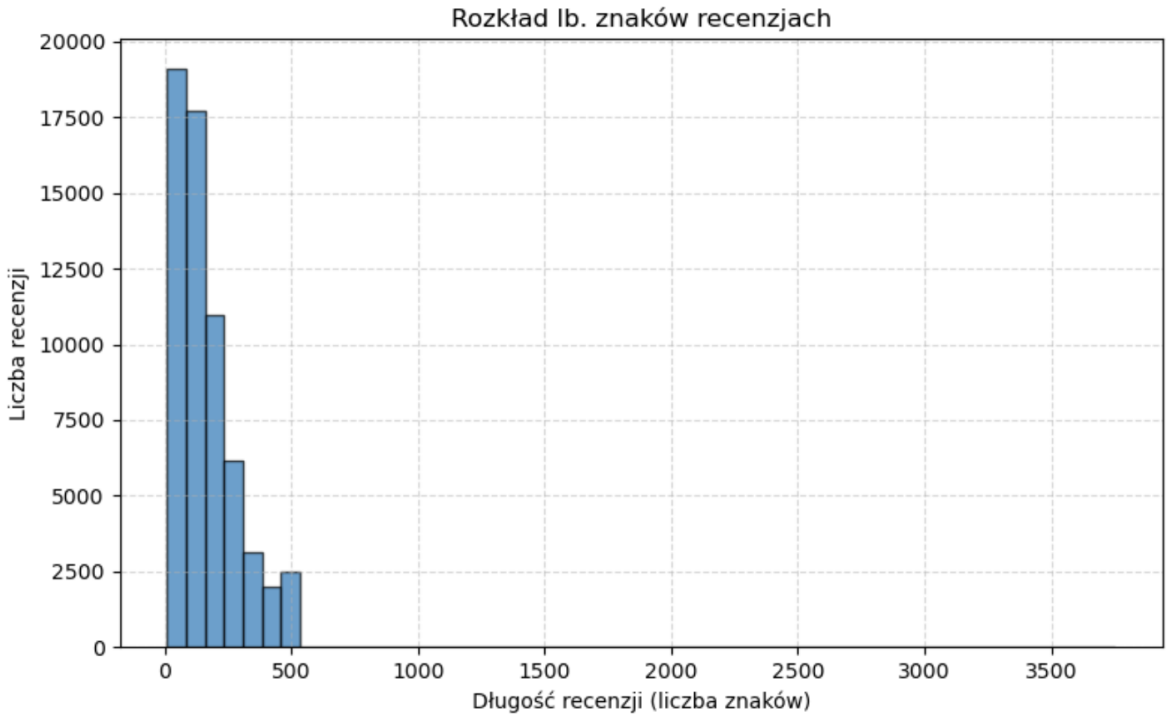
Time_submitted At what time the review was submitted	Review Review text	Rating Given score (1-5)	Total_thumbsup How many people found the review helpful	Reply Review reply
2022-01-01 2022-07-09	61356 unique values	1 5	0 8195	[null] 100% Hey, sorry to hear ... 0% Other (209) 0%
2022-07-09 15:00:00	Great music service, the audio is high quality and the app is easy to use. Also very quick and frien...	5	2	
2022-07-09 14:21:22	Please ignore	5	1	

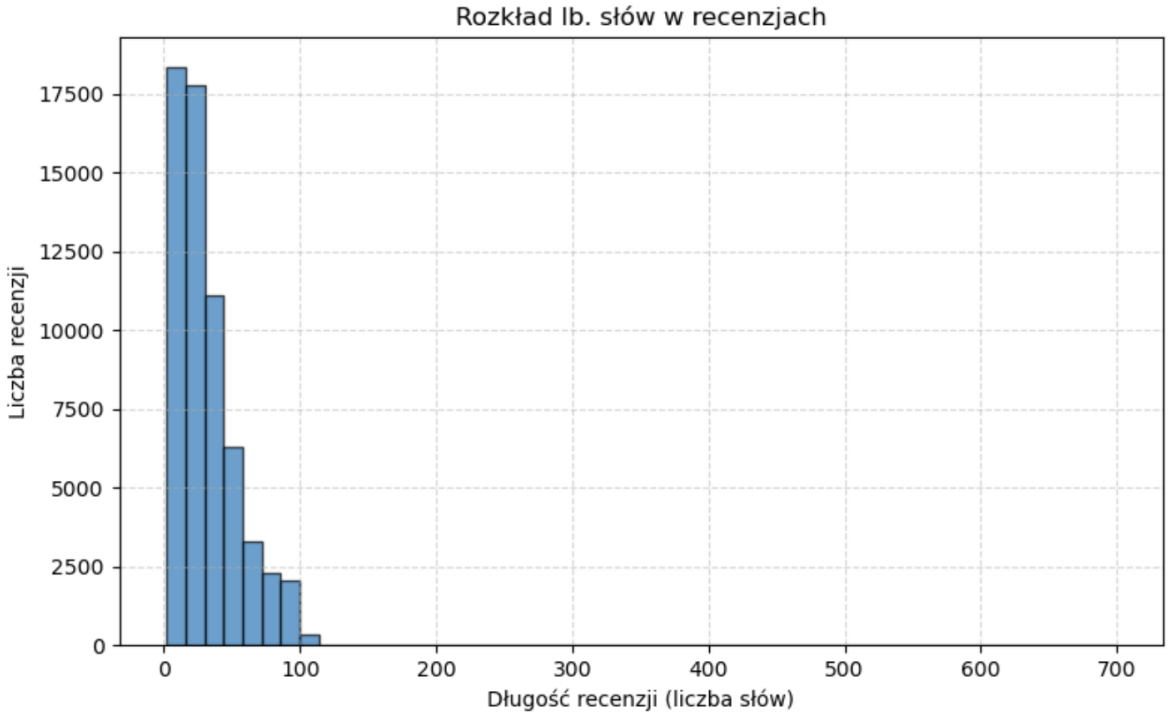
Eksploracja danych

- Rozkład ocen recenzji
- Rozkład liczby znaków w recenzjach
- Rozkład liczby słów w recenzjach
- Sprawdzenie czy występują recenzje bez treści
- Rozkład najczęściej występujących słów w korpusie

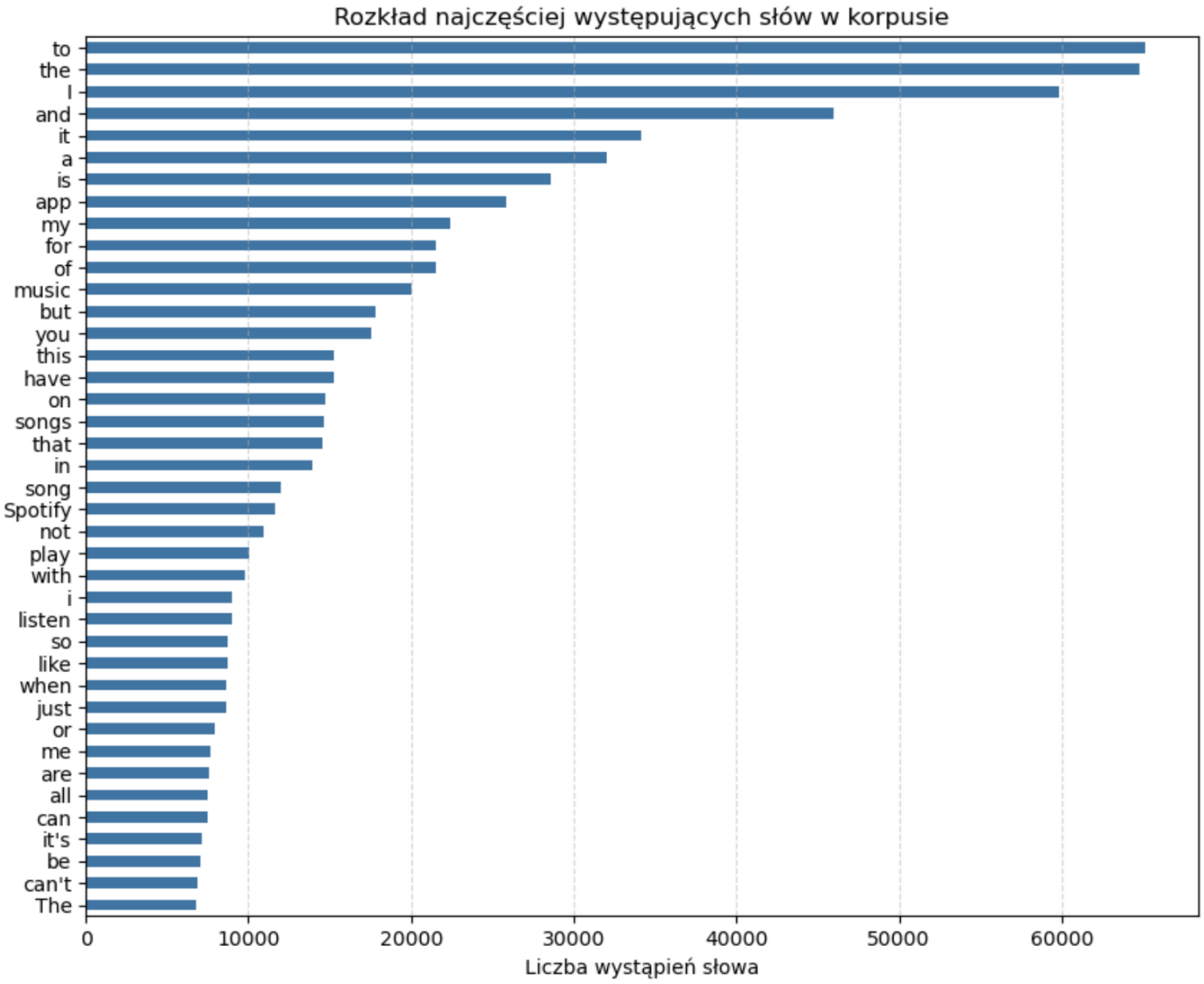


Eksploracja danych





Eksploracja danych



Przetwarzanie tekstu

- czyszczenie tekstu
- tokenizacja na słowa
- stemming i lametyzacja
- zdefiniowanie i usunięcie stopwordsów

Czyszczenie tekstu

- Usuwa znaczniki HTML
- Wyszukuje, konwertuje i dodaje na koniec przetworzonego tekstu zmodyfikowane emotikony
- Konwertuje tekst na małe litery
- Usuwa znaki, które nie są literami, cyframi, spacjami ani **apostrofami** i zastępuje spacją

```
Oryginalny tekst:
Dear Spotify why do I get songs that I didn't put on my playlist??? And why do we have shuffle play?


Po oczyszczeniu:
dear spotify why do i get songs that i didn't put on my playlist    and why do we have shuffle play
```

Tokenizacja na słowa

- Tokenizator word_tokenize z biblioteki NLTK – **problem**: rozdział słów z apostrofem

```
Przed tokenizacją na słowa:
dear spotify why do i get songs that i didn't put on my playlist    and why do we have shuffle play


Tokenizacja na słowa:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'songs', 'that', 'i', 'did', "n't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'p
lay']
```

- Tokenizator zdefiniowany wyrażeniem regularnym

```
Przed tokenizacją na słowa:
dear spotify why do i get songs that i didn't put on my playlist    and why do we have shuffle play


Tokenizacja na słowa:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'songs', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'pla
y']
```

Stemming i lametyzacja

- Stemming za pomocą PorterStemmer z biblioteki NLTK – **problem**: zamian 'y' na 'i'

```
Przed stemmingiem:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'songs', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'pla
y']
Po stemmingu:
['dear', 'spotifi', 'whi', 'do', 'i', 'get', 'song', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'whi', 'do', 'we', 'have', 'shuffl', 'play']
```

- Stemming za pomocą SnowballStemmer z biblioteki NLTK – **problem**: zamian 'y' na 'i'

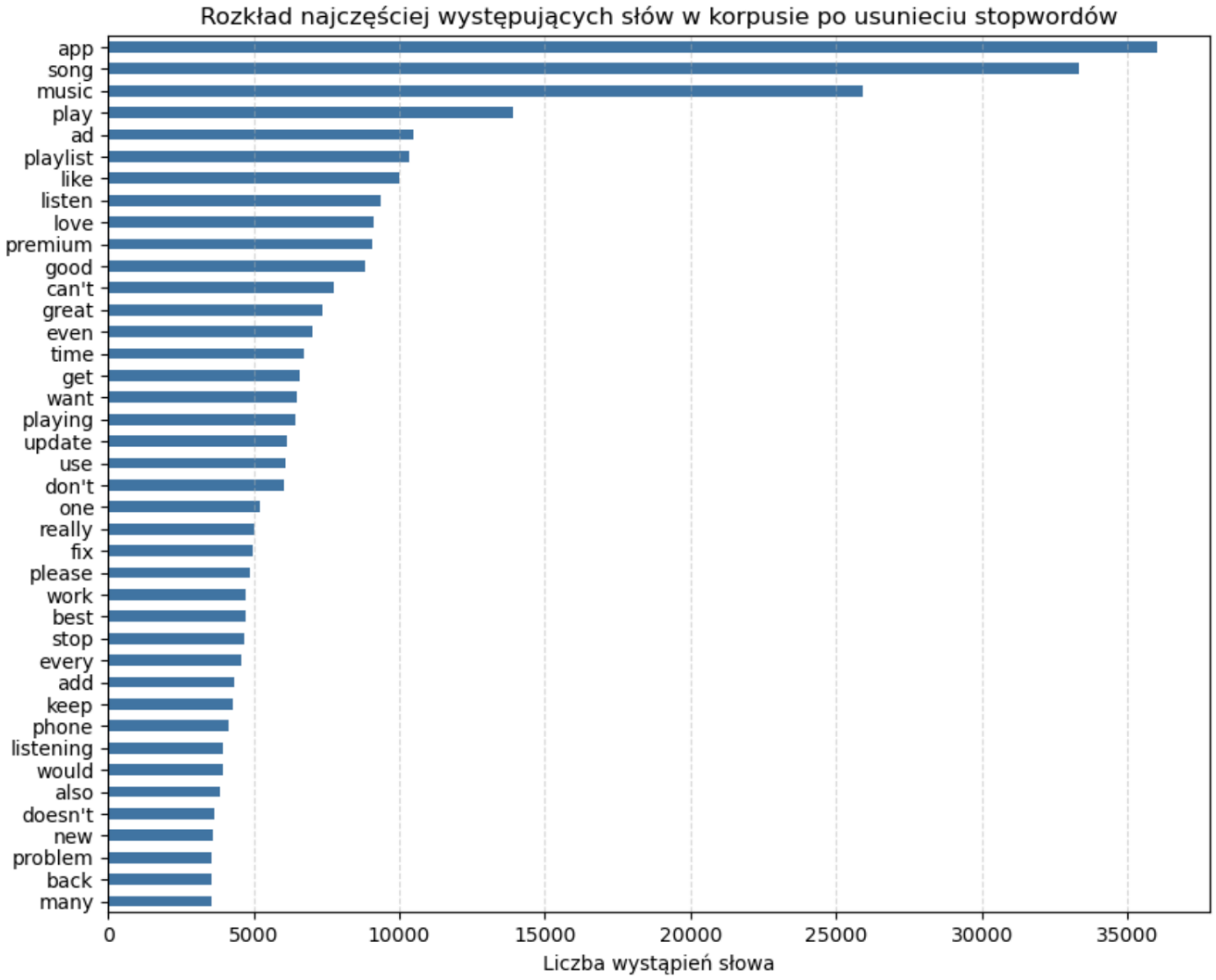
```
Przed stemmingiem:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'songs', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'pla
y']
Po stemmingu:
['dear', 'spotifi', 'whi', 'do', 'i', 'get', 'song', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'whi', 'do', 'we', 'have', 'shuffl', 'play']
```

- Lametyzacja za pomocą WordNetLemmatizer z biblioteki NLTK

```
Przed lametyzacją:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'songs', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'pla
y']
Po lametyzacji:
['dear', 'spotify', 'why', 'do', 'i', 'get', 'song', 'that', 'i', "didn't", 'put', 'on', 'my', 'playlist', 'and', 'why', 'do', 'we', 'have', 'shuffle', 'play']
```

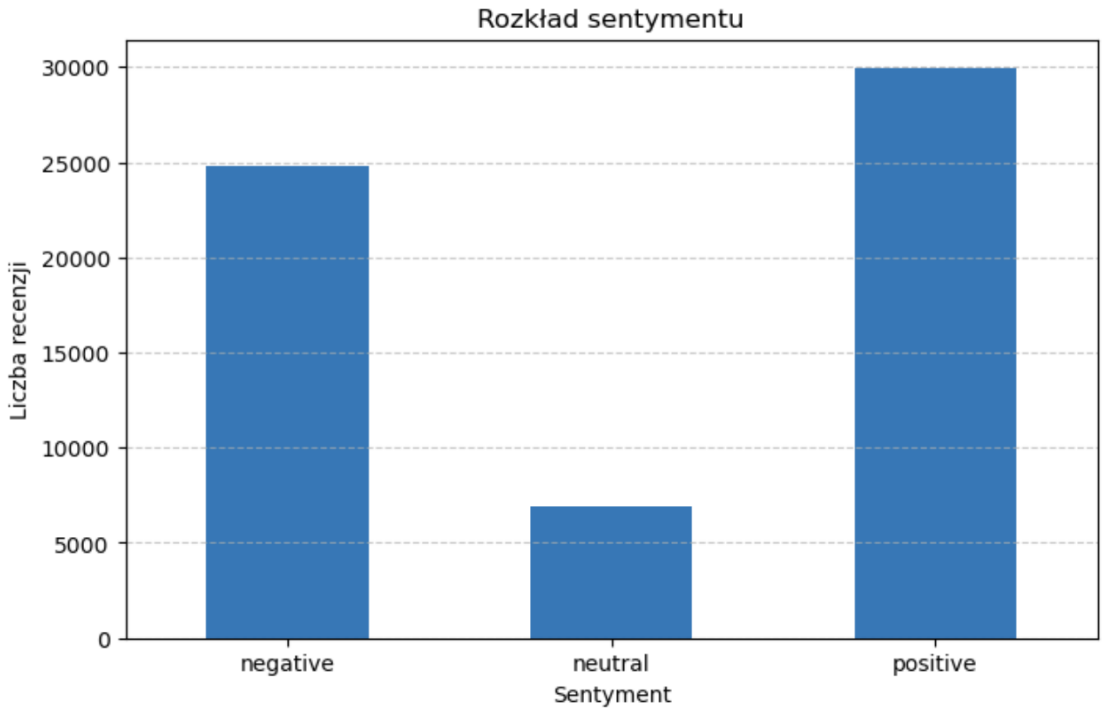
Zdefiniowanie i usnięcie stopwordsów

- Stworzenie korpusu po lametyzacji
- Modyfikacja listy stopwords z biblioteki NLTK:
  - Usunięcie z listy: ["don't", "didn't", "doesn't", "can't", "couldn't", "isn't"]
  - Dodanie do listy: ['spotify', '12', 'wa', 'ha', "i'm", "i've"]
- Usunięcie stopwordsów



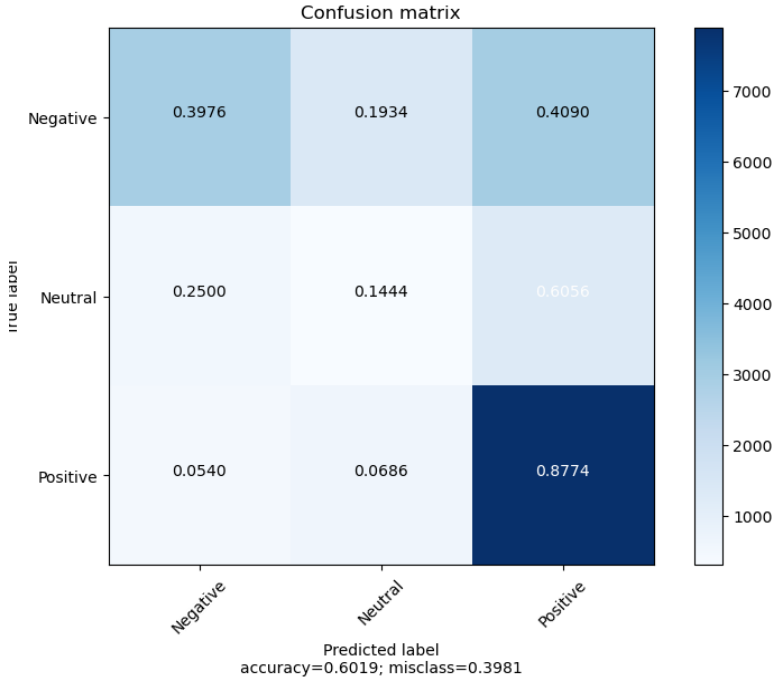
Przygotowania do modelowania

- Klasyfikacja opinii na podstawie ratingu w celu uproszczenia analizy wg kryteriów:
  - Pozytywne - Rating: 4-5
  - Neutralne - Rating: 3
  - Negatywne - Rating: 1-2
- Zdefiniowanie x i y:
  - X - dane tekstowe po oczyszczeniu, tokenizacji, lametyzacji i usunięciu stopwordsów
  - Y - sentyment przypisany po klasyfikacji na podstawie ratingu
- Podział zbioru na:
  - Część treningowa – 30 %
  - Część testowa – 70%



Analiza sentymentu - TextBlob

Macierz pomyłek dla danych testowych po usunięciu stopwordsów (przykład):



Predykcja sentymentu dla danych testowych na podstawie wartości polaryzacji tekstu obliczanej przez TextBlob:

Modyfikacje tekstu	Accuracy
Orginalny tekst	0.6138
Oczyszczony tekst	0.6154
Tekst po tokenizacji na słowa	0.6148
Tekst po lametyzacji	0.6128
Tekst po usunięciu stopwordsów	0.6018

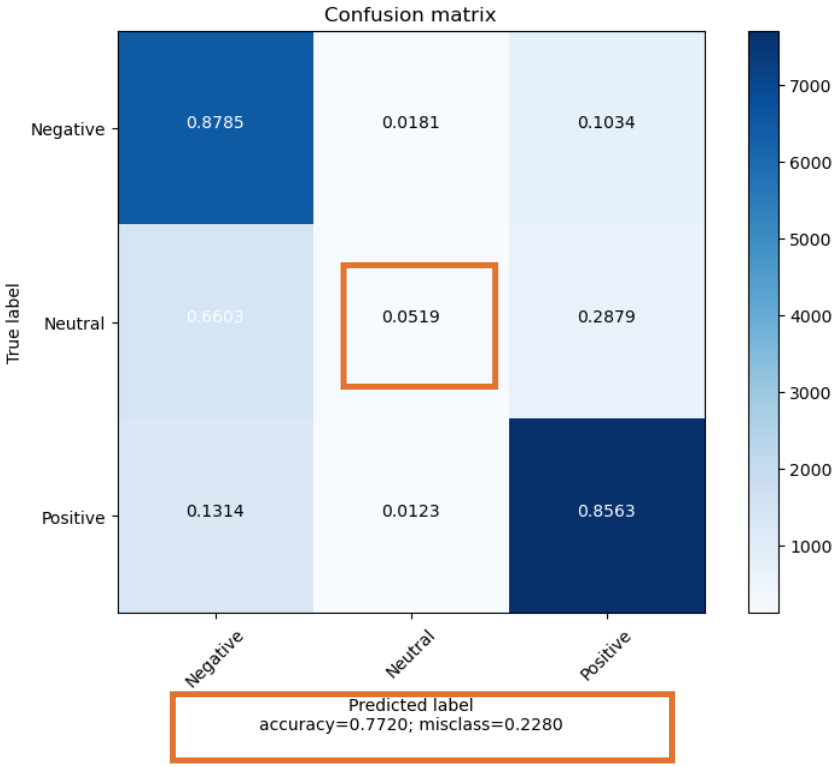
Dokładność predykcji nieznacznie się różni między danymi nieoczyszczonymi, a poddanym różnym zabiegom ich przetwarzania. Klasyfikacja Textblob dobrze klasyfikuje tylko pozytywne opinie.

Modelowanie – trenowanie własnego modelu

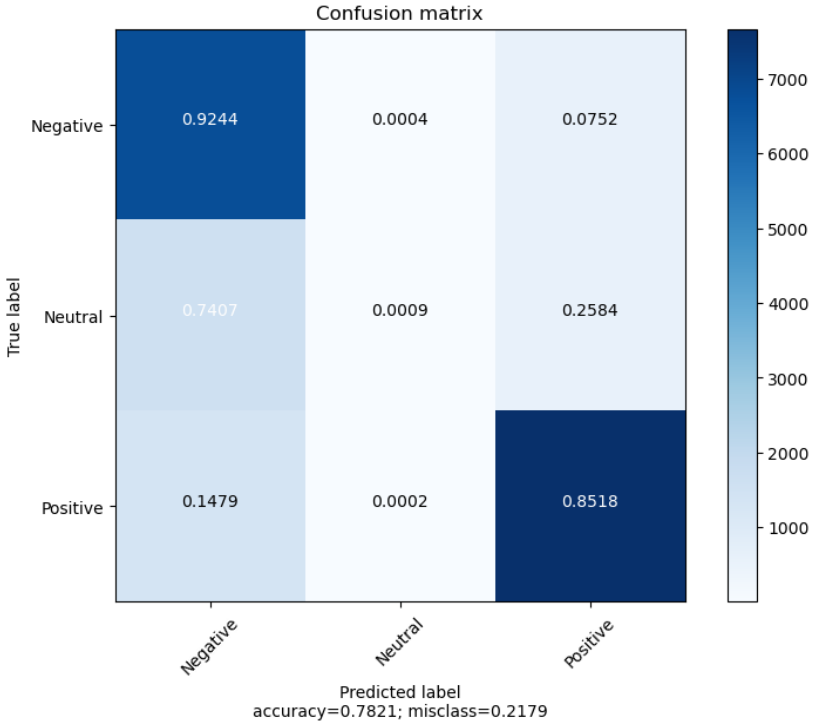
- Do klasyfikacji sentymentu wykorzystano:
  - Naiwny klasyfikator Bayesowski
  - Regresja logistyczna z multi_class='multinomial'
- Tekst do modeli był reprezentowany metodą:
  - Bag of Words (BoW)
  - N-gram
  - TF-IDF
- Modele oceniano na podstawie:
  - Accuracy dla zbioru testowego
  - Macierzy pomyłek
  - f1-score

Naiwny klasyfikator bayesowski

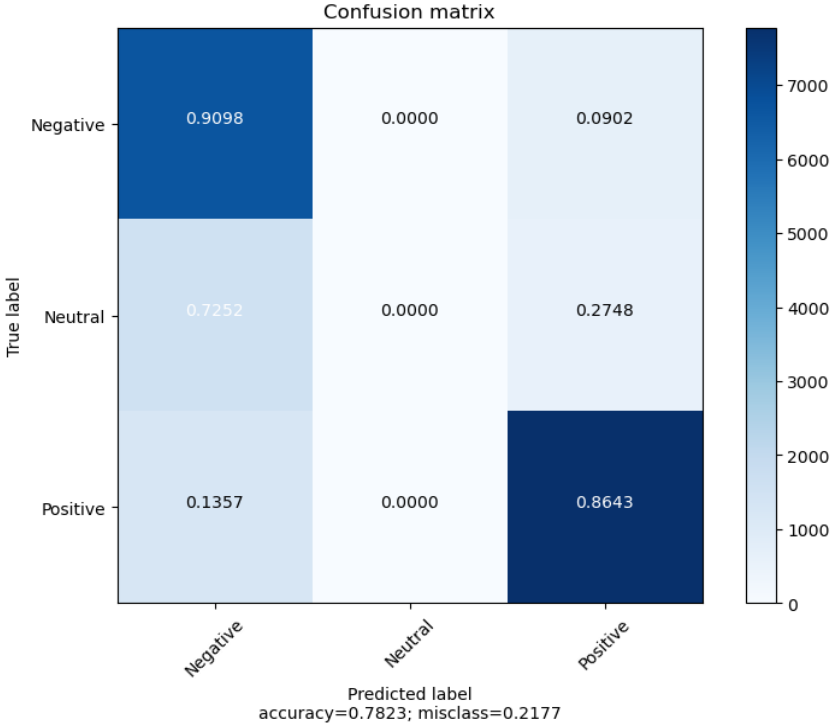
Reprezentacja tekstu - BoW



Reprezentacja tekstu – N-gram



Reprezentacja tekstu – TF-IDF



Naiwny klasyfikator bayesowski

Reprezentacja tekstu - BoW

Accuracy dla zbioru treningowego: 0.8036878116664734
Accuracy dla zbioru testowego: 0.7719573569998377

	precision	recall	f1-score	support
Negative	0.71	0.88	0.79	7342
Neutral	0.31	0.05	0.09	2140
Positive	0.85	0.86	0.85	8997
accuracy			0.77	18479
macro avg	0.62	0.60	0.58	18479
weighted avg	0.73	0.77	0.74	18479

Reprezentacja tekstu – N-gram

Accuracy dla zbioru treningowego: 0.8761219993041864
Accuracy dla zbioru testowego: 0.7821310676984685

	precision	recall	f1-score	support
Negative	0.70	0.92	0.80	7342
Neutral	0.29	0.00	0.00	2140
Positive	0.87	0.85	0.86	8997
accuracy			0.78	18479
macro avg	0.62	0.59	0.55	18479
weighted avg	0.74	0.78	0.74	18479

Reprezentacja tekstu – TF-IDF

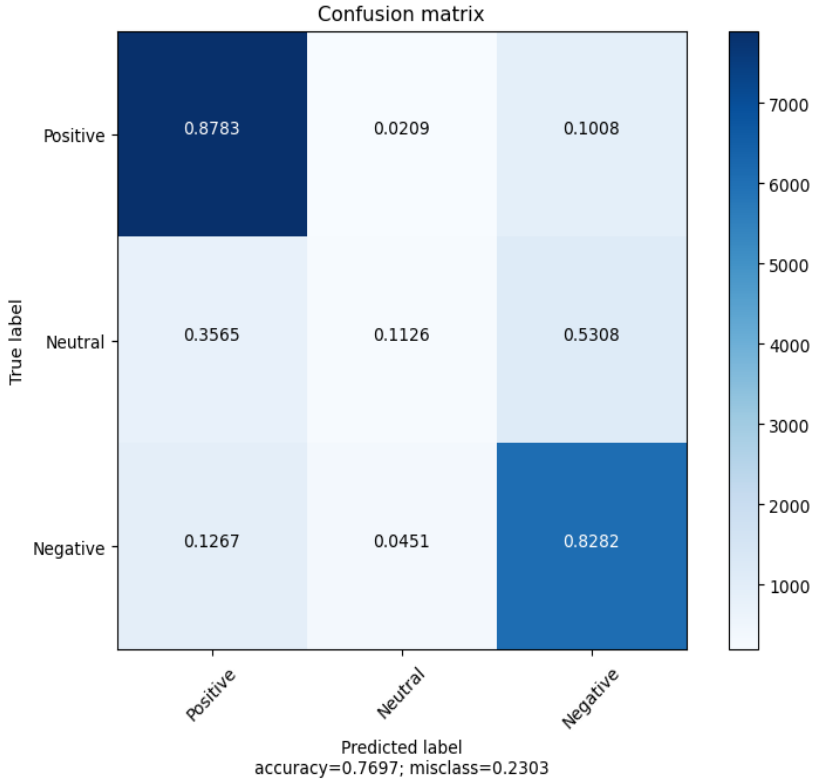
Accuracy dla zbioru treningowego: 0.8312188333526614
Accuracy dla zbioru testowego: 0.7822934141457871

	precision	recall	f1-score	support
Negative	0.71	0.91	0.80	7342
Neutral	0.00	0.00	0.00	2140
Positive	0.86	0.86	0.86	8997
accuracy			0.78	18479
macro avg	0.52	0.59	0.55	18479
weighted avg	0.70	0.78	0.74	18479

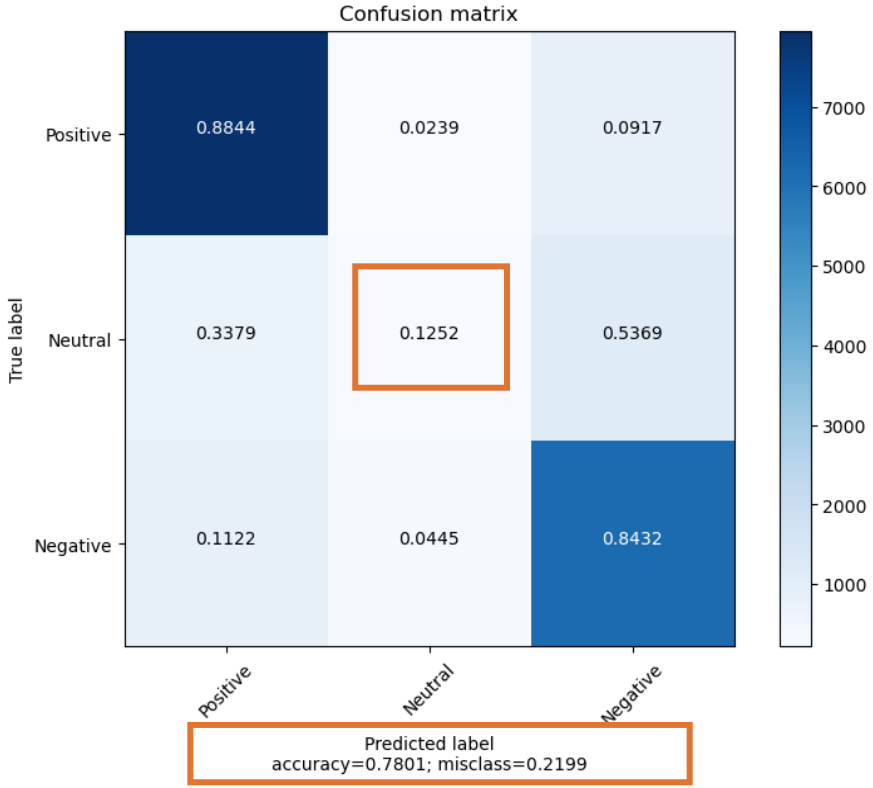
Najlepszym modelem bayesowskim wydaje się być ten, w którym tekst jest reprezentowany przez BoW, ze względu na to, że ma dość wysoką wartość accuracy (0,77) i jakkolwiek uwzględnia sentyment opinii neutralnych (macierz pomyłek, wartość f1-score).

Regresja logistyczna

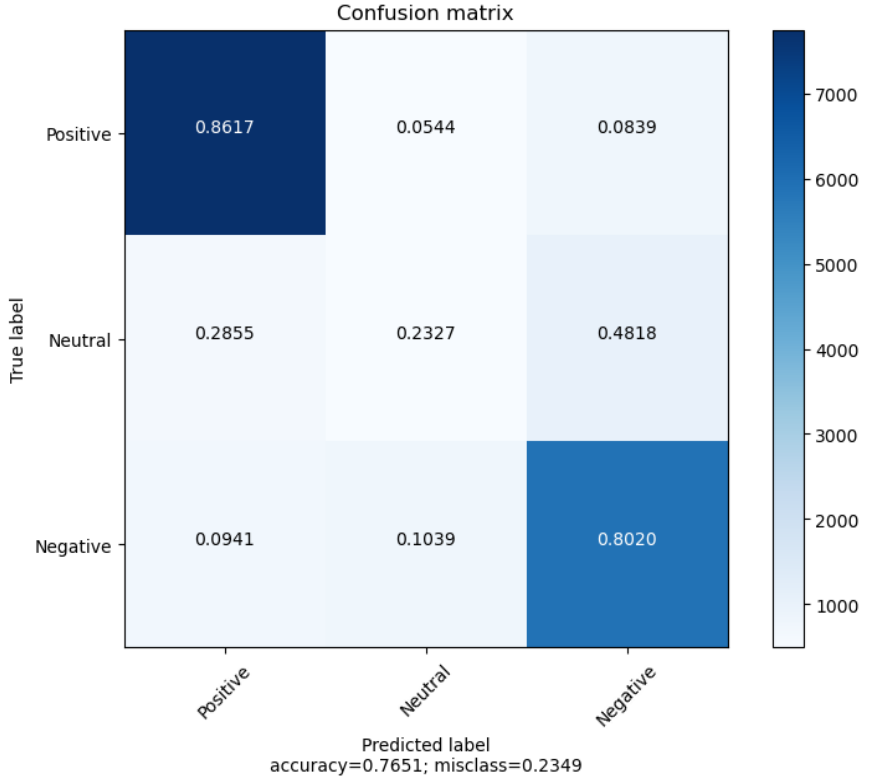
Reprezentacja tekstu - BoW



Reprezentacja tekstu – N-gram



Reprezentacja tekstu – TF-IDF



Regresja logistyczna

Reprezentacja tekstu - BoW

Accuracy dla zbioru treningowego: 0.851513394410298
Accuracy dla zbioru testowego: 0.769738622219817

	precision	recall	f1-score	support
Positive	0.82	0.88	0.85	8997
Neutral	0.32	0.11	0.17	2140
Negative	0.75	0.83	0.79	7342
accuracy			0.77	18479
macro avg	0.63	0.61	0.60	18479
weighted avg	0.74	0.77	0.75	18479

Reprezentacja tekstu – N-gram

Accuracy dla zbioru treningowego: 0.9899107039313464
Accuracy dla zbioru testowego: 0.7801287948482061

	precision	recall	f1-score	support
Positive	0.84	0.88	0.86	8997
Neutral	0.33	0.13	0.18	2140
Negative	0.76	0.84	0.80	7342
accuracy			0.78	18479
macro avg	0.64	0.62	0.61	18479
weighted avg	0.75	0.78	0.76	18479

Reprezentacja tekstu – TF-IDF

Accuracy dla zbioru treningowego: 0.8678418183926707
Accuracy dla zbioru testowego: 0.7904648519941555

	precision	recall	f1-score	support
Positive	0.86	0.88	0.87	8997
Neutral	0.47	0.05	0.09	2140
Negative	0.73	0.90	0.81	7342
accuracy			0.79	18479
macro avg	0.69	0.61	0.59	18479
weighted avg	0.76	0.79	0.75	18479

Najlepszym modelem regresji logistycznej wydaje się być ten, w którym tekst jest reprezentowany przez n-gramy, ze względu na to, że ma dość wysoką wartość accuracy dla danych testowych (0.78) uwzględniając przy tym najlepiej sentyment opinii neutralnych (macierz pomyłek, wartość f1-score).

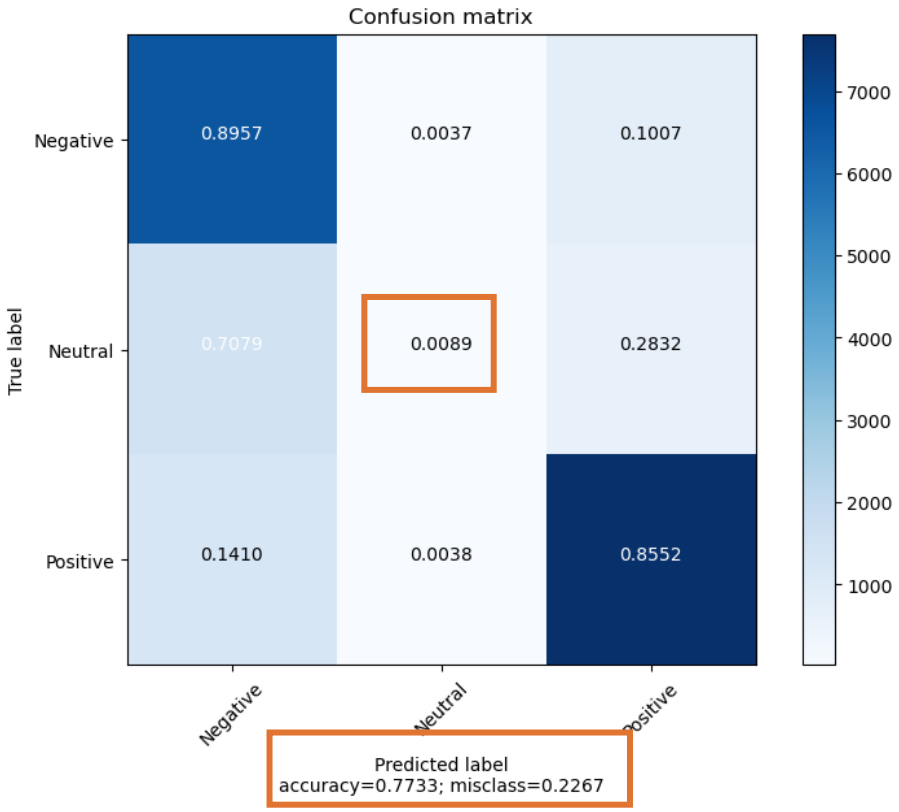
Modyfikacje najlepszych modeli

- Model bayesowski z reprezentacją tekstu za pomocą BoW:
  - znalezienie najlepszej wartości parametru alpha przy użyciu GridSearchCV zastosowaniem 3-krotnej walidacji krzyżowej
  - badany zakres wartości 'alpha' : [0.7, 1.0, 1.5, 2] -> alpha = 2

```
Accuracy dla zbioru treningowego: 0.7923692450423286
Accuracy dla zbioru testowego: 0.7732561285783863
              precision    recall  f1-score   support

    Negative       0.70      0.90      0.79      7342
     Neutral       0.24      0.01      0.02      2140
    Positive       0.85      0.86      0.85      8997

    accuracy                           0.77     18479
   macro avg       0.60      0.59      0.55     18479
weighted avg       0.72      0.77      0.73     18479
```



Zastosowane modyfikacje spowodowały zmniejszenie poprawności klasyfikacji opinii neutralnych (f1-score: 0.09 -> 0.02) bez znaczącej poprawy miar dla pozostałych klas.

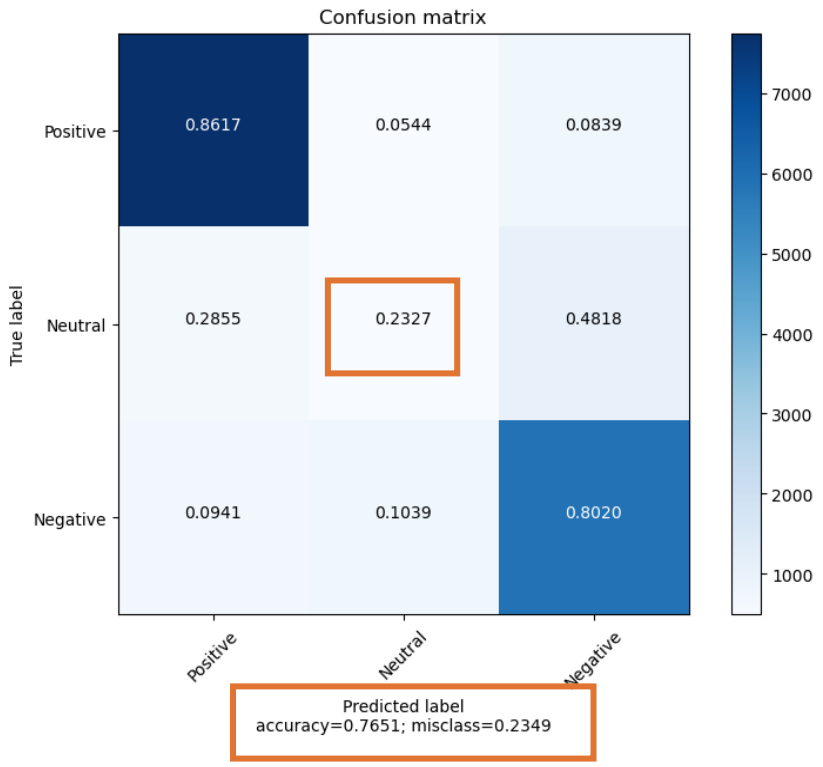
Modyfikacje najlepszych modeli

- Model regresji logistycznej z reprezentacją tekstu za pomocą N-gram:
  - balansowanie wag klas -> class_weight='balanced'
  - znalezienie najlepszej wartości parametru C przy użyciu GridSearchCV zastosowaniem 3-krotnej walidacji krzyżowej
  - badany zakres wartości 'C': [0.1, 0.5, 1, 3] -> C = 0.5

```
Accuracy dla zbioru treningowego: 0.9829525687115853
Accuracy dla zbioru testowego: 0.7651388062124573
              precision    recall  f1-score   support

    Positive       0.86      0.86      0.86      8997
     Neutral       0.28      0.23      0.26      2140
    Negative       0.77      0.80      0.78      7342

    accuracy                           0.77     18479
   macro avg       0.64      0.63      0.63     18479
weighted avg       0.75      0.77      0.76     18479
```



Zastosowane modyfikacje spowodowały zwiększenie poprawności klasyfikacji opinii neutralnych (f1-score: 0.18 -> 0.26) bez znaczącej zmiany miar dla pozostałych klas.

Wnioski

- TextBlob we wszystkich przypadkach daje podobny wynik accuracy – przetwarzanie tekstu nieznacznie wpływają na jego analizę sentymentu. Klasyfikacja na podstawie wartości polaryzacji najlepiej się sprawdza do klasyfikowania opinii pozytywnych, negatywne i neutralne bardzo słabo klasyfikuje
- Modele bayesowskie i regresji logistycznej dobrze klasyfikują pozytywne i negatywne opinie jednak słabo sobie radzą z neutralnymi
- Modele z reprezentacją tekstu typu TF-IDF najgorzej sobie radzą z klasyfikacją opinii neutralnych
- Najlepsze wyniki uzyskał zmodyfikowany model regresji logistycznej z reprezentacją tekstu w postaci n-gram. Niższa skuteczność dla klasy neutralnej wynika prawdopodobnie z niejednoznaczności językowej oraz trudności w rozróżnieniu opinii o mieszanym wydźwięku

Co dalej?

- Szczegółowa analiza opinii neutralnych w celu zwiększenia poprawności ich klasyfikacji przez modele
- Zastosowanie wag w modelach bayesowskich – MultinomialNB() nie posiada wbudowanego parametru automatycznie balansującego wagi klas, trzeba je ustawić ręcznie
- Modyfikacja innych parametrów modeli w celu ich optymalizacji
- Zmiana klasyfikacji sentymentu - zwiększenie liczebności opinii neutralnych poprzez inny podział sentymentu na podstawie ratingu